Shape the Code, Not the Proof

Detailed project plan, divided among three team members

Small language models (1.5B to 3B parameters) are poor at writing Lean 4 proofs. This project tests whether they do better when they only repair the **code** and Lean's built-in automatic tactics do the proving. The comparison of "model writes the proof" (condition B) against "model shapes the code, automation proves it" (condition C) is the whole paper.

How this document is organised

Section	Content
1	Goal, research questions, and what counts as success
2	Shared experimental design (frozen rules every member follows)
3	How the work is split, interface contracts, and the shared timeline
Part 1 (Member A)	Verification Engine: everything that touches Lean
Part 2 (Member B)	Tasks and Models: bugs, datasets, local models, prompts, repair loops
Part 3 (Member C)	Study Lead: related work, pre-registration, experiment driver, statistics, paper
4	Joint milestones, go/no-go gate, backup study
5	Risks, open decisions, and publication strategy

Each Part is one contiguous block of work owned end to end by one person. The three blocks meet only at two small interfaces (a task manifest and an attempt log), so members can work in parallel without waiting on each other after Week 1.

1. Goal and research questions

Idea in one line. Do not make small models write Lean proofs. Have them fix the code so that Lean's automatic provers (`simp`, `omega`, `grind`, and others) can do the proof.
Why it matters. In Lean you write code and a proof that the code meets its specification. Even strong models mostly fail at the proof part: on VERINA the best model wrote correct code far more often than it wrote valid proofs (check the exact figures against the paper before quoting them; see open decisions). Laptop-sized models are worse, yet they are the ones people can run cheaply and privately.

Research questions.

ID	Question	Hypothesis
RQ1 (main)	At equal budget, does C (repair code, automation proves it) solve more tasks than B (write code and proof with feedback)?	H1: C success rate is higher than B. H0: no difference or B higher.
RQ2	Does C find fixes that B misses, and the reverse? Report solved by both, only C, only B, neither.	C solves a non-empty set that B does not.
RQ3	Which tactic closes each proof, and which code shapes make automation succeed?	A small catalogue of shapes explains most successes.
RQ4	Does the stuck-point hint help versus a plain retry?	Hint improves C over C-plain.
RQ5 (optional)	How does the B versus C gap change from 1.5B to 3B to 7B?	Gap shrinks as models grow.

A clean negative result (B at least as good as C) is still reported. The contribution is the measured comparison, not a guaranteed win.

Novelty position

A literature search on 2026-10-29 found no paper that tests this exact comparison for small models in Lean. Close neighbours that a reviewer will raise: Dissect-and-Restore (code restructuring for Dafny, not Lean, not small models), APRIL and APOLLO (Lean proof repair, not code repair), the Lean-Prover agent (hammer pre-pass, not a paper), Goedel-Code-Prover (large models, proof search), VeriContest and the vericoding benchmark (benchmarks). Absence in four searches is not proof of absence, so Member C owns a repeating novelty check (Part 3).

What counts as success of the project

- A complete, frozen, auditable experiment with a measured B versus C result at equal budget.
- A buggy-program set derived from VERINA (VERINA does not ship one), released with provenance.
- A catalogue of code shapes that make Lean automation succeed.
- A workshop-length paper (LLM4Code or FormaliSE first). If C clearly beats B, aim for ASE NIER or the main track.

2. Shared experimental design (frozen rules)

Everything here is agreed by all three members in Week 1 and written into the pre-registration folder. Nobody changes it after the Week 2 freeze without telling the other two.

2.1 Definitions

Term	Definition
Task	A VERINA problem with a locked natural-language description, formal spec (pre and postcondition), hidden tests, and a buggy implementation.
Buggy program	Compiles, yet violates the spec on at least one test. Built by mutating VERINA reference solutions (operator swap, off-by-one, wrong bound, dropped case, swapped branch), kept only if it compiles and fails a test. Optionally also real wrong outputs from small models. Bug type is recorded per task.
Repair success (end to end)	All four hold: (1) new code compiles; (2) locked spec theorem proved with no `sorry`, no new `axiom`, no disallowed `native_decide`; (3) hidden tests pass; (4) spec and test files byte identical to originals by hash.
Budget	Equal across conditions: N model generations per task (primary). Tokens and wall clock reported as secondary. Lean automation time reported separately with a per-attempt cap. Suggested N = 5, frozen after the pilot.

2.2 Conditions

Cond.	Model writes	Feedback	Proof comes from
A	code + proof	none (N independent samples, pass@N)	the model
B	code + proof	Lean error messages, up to N sequential tries	the model
C	code only	template hint naming where the prover got stuck, up to N tries	fixed tactic pipeline
C-plain	code only	only "failed, retry"	fixed tactic pipeline
T (reference)	nothing	none	tactic pipeline on the buggy code (must fail) and on the correct reference (upper bound for automation)

2.3 Models

- Primary: Qwen2.5-Coder-1.5B-Instruct and Qwen2.5-Coder-3B-Instruct, local, open weights, no paid APIs.
- Optional scaling: Qwen2.5-Coder-7B-Instruct (GPU server) and one non-Qwen small model such as Llama-3.2-3B-Instruct or Phi-3-mini.
- Fixed decoding (temperature, top_p, max tokens, seed list), same prompt length budget across A, B, C. Record exact model hash and quantization.

2.4 Fixed tactic pipeline (pre-registered)

Tried in this order, each with its own timeout: `simp`, `simp_all`, `omega`, `decide`, `grind` (needs Lean 4.24 or later), `aesop`, then combinations such as unfold then simp, unfold then omega, split then simp_all. The list and order are tuned only on the 20 pilot tasks, then frozen in a dated file with a hash. Pilot tasks are excluded from the final analysis. No per-task tactic customisation and no model-chosen tactics in C. Lean and Mathlib versions are pinned.

2.5 Integrity checks (run on every attempt)

1. Spec and test files hash-matched against the originals.
2. Scan and `#print axioms` for sorry, axiom, native_decide, implemented_by, unsafe.
3. Region diff: only the allowed implementation region may change.
4. The model never sees tests or the reference solution (not in the prompt, not in the container).
5. Every prompt, output, and Lean log is stored.

6 A 10 percent hand audit of reported successes for spec-gaming (for example a trivially satisfied spec).

2.6 Metrics and statistics

- Primary: end-to-end repair success rate per condition and per model.
- Secondary: tasks solved by one condition only, attempts to success, compile rate, proof-validity rate (A and B), automation coverage, time and tokens per solved task.
- Paired tests since tasks are shared: McNemar for B versus C, bootstrap 95 percent confidence interval on the difference, effect size, at least 3 seeds per condition. No post-hoc subset picking.
- Breakdowns by bug type, difficulty, closing tactic, and code feature. 15 to 20 qualitative case studies.

3. How the work is split

The split follows the data flow, not the calendar. Each member owns one stage of the pipeline end to end, so every Part is a single contiguous block with no interleaved tasks:

	Part 1: Member A	Part 2: Member B	Part 3: Member C
Block name	Verification Engine	Tasks and Models	Study Lead
Owns	Everything that runs Lean: toolchain, VERINA build, tactic pipeline, hints, integrity checks, condition T	Everything that creates or runs model text: buggy tasks, task manifests, model serving, prompts, code generation loops for A, B, C	Everything around the experiment: related work, pre-registration, driver, statistics, figures, paper, artifact
Produces	`verify()` service and hint extractor	Task manifests and `generate()` service	Experiment driver, results, analysis, manuscript
Consumes	Task manifest, candidate code or proof	Verify result (for feedback), frozen prompts	Both services, all logs
Main skills	Lean 4, tactics, build systems	Python, LLM serving, mutation testing, prompting	Statistics, writing, experiment design
Share of effort	about 33 percent (heavy early)	about 33 percent (heavy early to mid)	about 33 percent (light early, heavy late)

3.1 Interface contracts (agreed and frozen in Week 1)

Only two things cross between Parts, so these must be fixed on day one. Member C writes them into `prereg/interfaces.md` and all three sign off.

Task manifest (JSON, one per task, written by B, read by A and C)

Field	Meaning
task_id, description	Identifier and natural-language statement shown to the model
spec_lean, tests_path	Locked spec file and hidden tests (never given to the model)
buggy_lean, reference_lean	Buggy implementation (shown to the model) and correct reference (A and C only)
bug_type, difficulty	Mutation category and VERINA difficulty, for breakdowns
split	pilot, main, or heldout
hashes	SHA-256 of spec and tests for the integrity check

Attempt record (JSONL, one line per attempt, written by C's driver from A and B outputs)

Field	Meaning
task_id, condition, model, seed, attempt_idx	Which run this is
prompt_hash, model_output, extracted_code, extracted_proof	What the model produced (proof empty in C)
compile_ok, proof_ok, closing_tactic, tests_pass, integrity_ok	Verification outcome (from A)
hint	Stuck-point hint sent back (C and C-plain only)
tokens_in, tokens_out, model_secs, lean_secs	Cost accounting

Two function calls (Python)

Call	Owner	Behaviour
`verify(task, code, proof=None, mode)`	A	mode is tactics or model_proof. Returns compile, proof, closing tactic, tests, integrity flags, and a hint object. Deterministic and cached.

Call	Owner	Behaviour
`generate(task, condition, history, seed)`	B	Returns the next candidate (code, or code plus proof) for the condition, using fixed decoding and the frozen prompt template.

The driver (Member C) loops over task, condition, model, seed, and attempt, calling `generate` then `verify`, writing one attempt record each time. Because the contract is fixed, A and B can each build against stub versions of the other's call during Week 1 and 2.

3.2 Shared timeline at a glance

Week	Dates	Part 1 (A)	Part 2 (B)	Part 3 (C)
1	from 11-02	Lean and VERINA up; verify() stub	Models served; mutation prototype	Related work; interfaces; definitions
2	from 11-09	Tactic pipeline, hints, integrity; T runs	20 pilot tasks; A/B/C generators	Driver v0; pilot run; go/no-go prep
3	from 11-16	Hardening, integrity tests, caching	60 to 100 tasks; held-out split	Driver v1; dry run; prereg freeze
4	from 11-23	Support runs; triage Lean failures	Support runs; triage model failures	Run main study; monitor
5	from 11-30	Tactic and code-shape catalogue (RQ3)	Optional 7B and extra model runs	Statistics; overlap; audit
6	from 12-07	Held-out run support; artifact (Lean side)	Ablations; artifact (data side)	Novelty re-check; draft paper
7	from 12-14	Review methods text	Review methods text	Final paper; replication package

PART 1 (Member A): Verification Engine. Everything that touches Lean.

Mission

Build and own the trusted judge. Given a task and a candidate, `verify()` must say, reliably and reproducibly, whether the repair is a true end-to-end success. The paper's credibility rests on this component, so correctness and integrity matter more than speed.

Scope (one contiguous block)

- Lean toolchain, Mathlib, and VERINA build, pinned.
- Fixed tactic pipeline and condition T.
- Stuck-point hint extractor.
- Integrity checker.
- Test runner for hidden tests.
- RQ3 analysis: which tactic and which code shape closed each proof.

Detailed tasks by week

Week	Task	Detail and acceptance check
1	Install and pin toolchain	1. Install elan. 2. Pin a Lean version with `grind` (4.24 or later) in `lean-toolchain`. 3. Fetch Mathlib via lake and commit `lake-manifest.json`. Check: clean build from scratch on a second machine.
1	Build VERINA	Clone VERINA, check out the pinned commit, make every reference solution compile, and run its tests. Record how many reference solutions compile and run. Check: a table of tasks with pass or fail.
1	verify() stub	Implement the contract returning fixed dummy values so B and C can integrate immediately. Check: B and C call it successfully.
2	Tactic pipeline	Implement the ordered tactic list with a timeout per tactic and a global cap. Return which tactic closed the goal. Check: runs on all reference solutions; report coverage (this is the automation upper bound, condition T).
2	Hint extractor	Parse the first unsolved goal and the tactic that got furthest. Add a rule-based pointer (non-structural recursion, Int and Nat mixed, opaque helper, partial def). No LLM. Check: hints are deterministic and under a length cap.
2	Integrity checker	Hash match for spec and tests, `#print axioms` scan, forbidden-keyword scan, region diff. Check: unit tests with injected sorry, axiom, weakened spec, edited test, edit outside region. All must be rejected.
2	Condition T runs	Run the pipeline on the buggy code (must fail) and on the reference (upper bound). Feed numbers to the go/no-go gate.
3	Harden and cache	Content-hash caching of results, parallel workers, crash recovery, resource limits so a runaway tactic cannot hang the machine. Check: 1000 verify calls complete unattended.
3	Test runner	Run hidden tests on candidate code via `#eval` or compiled executable. Check: agrees with VERINA's own result on all references.
4	Support the main run	On call: triage any Lean failure, timeouts, or non-determinism. Fix and tell C before a rerun.
5	RQ3 catalogue	For every successful C fix, label the code-shape change (structural recursion, if over match, avoiding Int and Nat mixing, inlining helpers, and so on) and the closing tactic. Produce a table and 10 to 15 annotated before and after examples.
6	Artifact (Lean side)	Dockerfile or script to rebuild the Lean environment; README for the verifier.
7	Paper support	Review Method and Threats sections for Lean accuracy.

Deliverables

- Pinned Lean environment, buildable from scratch.

- `verify()` with a test suite for the integrity checks.
- Frozen tactic list and hint templates (with C, hash in prereg).
- Condition T coverage table.
- RQ3 code-shape catalogue.

Definition of done

- Zero false successes in the integrity test suite.
- Same input gives the same output across reruns.
- Automation coverage on references is reported honestly, whatever the number.

Interfaces with others

- From B: task manifests. To B: verify results for feedback (Lean errors for B, hints for C).
- To C: attempt-level verification fields and coverage numbers for statistics.

Main risks for this Part

- Tactics prove too few programs. Mitigation: widen the tactic list on pilot tasks only; escalate at the Week 2 gate.
- Lean build or version drift. Mitigation: pin everything and use a lockfile.
- Slow tactics. Mitigation: per-tactic timeouts and caching.

PART 2 (Member B): Tasks and Models. Everything that creates or runs model text.

Mission

Create the buggy-program task set and run the small models faithfully under fixed decoding, so that conditions A, B, and C differ only in what the design says they differ in. The task set is itself a contribution because VERINA does not ship buggy programs.

Scope (one contiguous block)

- Mutation pipeline and the buggy task set, with manifests and splits.
- Local model serving with pinned weights and fixed decoding.
- Prompt templates for A, B, C and C-plain.
- The `generate()` service that produces the next candidate for any condition.
- Optional: real wrong outputs from small models as a second bug source.

Detailed tasks by week

Week	Task	Detail and acceptance check
1	Serve the models	1. Set up Ollama, llama.cpp, or vLLM. 2. Pull Qwen2.5-Coder 1.5B and 3B, record hashes and quantization. 3. Fix temperature, top_p, max tokens, and a seed list. Check: same prompt and seed returns identical text on rerun.
1	Mutation prototype	Operators: arithmetic swap, comparison flip, off-by-one on bounds, dropped branch, wrong initial value, swapped arguments. Check: 5 hand-verified mutants that compile and fail a test.
1	generate() stub	Return canned text under the contract so C's driver can integrate.
2	Pilot task set (20)	Stratify by VERINA difficulty and bug type. Keep only mutants that compile and fail at least one test (uses A's test runner). Write 20 manifests with hashes. Check: A can verify all 20 references and C can load all 20 manifests.
2	Prompt templates	Write prompts for A, B, C, C-plain with equal length budgets. C and C-plain show code only and never ask for a proof. Include the lemma-free rule: the model must not edit spec or tests. Check: character and token length table across conditions.
2	Real generate()	A: N independent samples. B: sequential, feeds Lean errors from verify(). C: sequential, feeds the hint. C-plain: feeds only a retry message. Extract code and proof from model text robustly (code fences, partial output).
3	Scale the task set (60 to 100)	Write the selection rule before sampling: filter, stratified random sample, fixed seed. Lock a roughly 30 percent held-out split that nobody inspects until the final run. Check: manifest counts by split, bug type, and difficulty.
3	Model robustness	Handle empty output, very long output, repeated text, timeouts. Log every failure rather than dropping it.
3	Dry run support	With C, dry run 5 tasks across all conditions and fix extraction or prompt bugs.
4	Support the main run	On call: triage model failures, out-of-memory, extraction errors. Rerun only infrastructure failures, never model failures.
5	Extended models (optional)	7B on the GPU server (never sudo reboot it), and one non-Qwen small model. Same prompts and decoding.
5	Real-error bug source (optional)	Collect actual wrong small-model outputs on VERINA code generation as a second bug set; report mutant and natural results separately.
6	Ablations	Hint variants and sensitivity to N, run through the same driver.
6	Artifact (data side)	Release task manifests, mutation scripts, provenance table, prompts.
7	Paper support	Review Task Construction and Models sections.

Deliverables

- Task manifests for pilot, main, and held-out splits with provenance and hashes.
- Mutation scripts and a bug-type table.
- Frozen prompt templates and decoding config.
- `generate()` with tests.
- A short note on mutant realism for the Threats section.

Definition of done

- Every task is verifiably buggy and verifiably fixable (the reference passes).
- Model outputs are reproducible for a given seed.
- Conditions differ only by design (prompt length table proves it).

Interfaces with others

- To A and C: task manifests. From A: verify results used as feedback inside the loops.
- To C: attempt-level model fields (text, tokens, model time).

Main risks for this Part

- Mutants unrealistic. Mitigation: add the real-error bug source and report both.
- Small models fail even at simple repair. Mitigation: report compile rate; condition T separates model failure from automation failure.
- Compute limits. Mitigation: start with 60 tasks; use the GPU server only for 7B.

PART 3 (Member C): Study Lead. Related work, pre-registration, driver, statistics, paper.

Mission

Own the scientific story and the glue. Make sure the novelty claim is defensible, the rules are frozen before the main run, the experiment executes exactly as registered, and the results become a clear paper. This block is light early and heavy late, so it also carries the novelty checks that run alongside everyone else's engineering.

Scope (one contiguous block)

- Related work and the novelty matrix, repeated before writing.
- Pre-registration folder and interface contracts.
- Experiment driver that wires A's verify() to B's generate().
- Analysis: statistics, overlap, breakdowns, figures, hand audit.
- Paper, replication package, and coordination with the professor.

Detailed tasks by week

Week	Task	Detail and acceptance check
1	Read and map related work	Read in full: Dissect-and-Restore, APRIL, APOLLO, the Lean-Prover agent, Goedel-Code-Prover, VERINA, the vericoding benchmark, LeanHammer, DafnyFix, Laurel. Start `related_work/novelty_matrix.md` with columns: repairs code or proof, language, model size, proof-writing baseline present, automation used, our difference.
1	Targeted searches	Log query and date for each: proof-friendly code; verification-aware refactoring with LLMs; hammer plus LLM verified code generation; small language model Lean 4; tactic automation code repair; also citations of VERINA, Dissect-and-Restore, APRIL. Ask the professor for pointers.
1	Interfaces and definitions	Write `prereg/interfaces.md` (manifest, attempt record, two calls) and `prereg/definitions.md` (section 2 of this plan). Get sign-off from A and B.
2	Driver v0	Loop over task, condition, model, seed, attempt; call generate() then verify(); append one JSONL record. Resume after a crash, skip finished work, parallel workers. Check: runs end to end on stubs.
2	Pilot run	Run T, A, B, C once on 20 pilot tasks with one model. Collect numbers for the gate.
2	Go/no-go meeting	Evaluate the four criteria in section 4 with the professor. If no-go, choose a backup study. Freeze tactics, hint templates, prompts, seeds, N with hashes in `prereg/`.
3	Driver v1	Add cost accounting, integrity-failure handling, deterministic ordering, and a one-command full run. Check: 5-task dry run across all conditions with A and B.
3	Cost estimate	Compute GPU hours for tasks x conditions x seeds x models and confirm it fits; trim tasks to 60 first if not.
4	Run the main study	Launch A, B, C, C-plain for 1.5B and 3B, 3 seeds. Daily check of progress and failures; rerun only infrastructure failures.
5	Statistics	McNemar for B versus C, bootstrap 95 percent CIs, effect sizes, overlap diagram (both, only C, only B, neither), per-bug-type and per-difficulty tables. Plots with consistent colours.
5	Hand audit and case studies	Audit 10 percent of successes for spec-gaming. Pick 15 to 20 case studies with A's catalogue.
6	Held-out and robustness	Run the held-out split exactly once. Report sensitivity to N and to the hint.
6	Novelty re-check	Repeat all searches. If someone published first, reposition (for example around small models or the code-shape catalogue) before writing.
6	Draft paper	Introduction, Related Work, Method, and Setup. Lock the claims to what the numbers support.

Week	Task	Detail and acceptance check
7	Finish paper and artifact	Results, Discussion, Threats, Limitations. Replication package with hashes. Professor review, revise, submit or hold for the venue deadline.

Paper outline (workshop length, 4 to 8 pages)

1 Introduction: small models cannot write Lean proofs; claim and contributions.
2 Background and related work (novelty matrix condensed).
3 Approach: shape the code, fixed tactics, hint loop.
4 Experimental setup: tasks, bug construction, conditions, budgets, integrity.
5 Results: RQ1 to RQ4 (RQ5 if done).
6 Analysis: which code shapes make proofs automatic; case studies.
7 Threats to validity: mutation realism, VERINA scope, tactic list generality, quantization, Lean version.
8 Conclusion and artifact.

Contributions to claim: (1) a measured B versus C comparison for small models, (2) a buggy-program set derived from VERINA, (3) a frozen auditable repair-and-verify harness, (4) a catalogue of proof-friendly code shapes.

Deliverables

- Novelty matrix and related-work text.
- Frozen `prereg/` with hashes.
- Driver and raw results.
- Statistics scripts, figures, tables.
- Paper and replication package.

Definition of done

- Every number in the paper regenerates from raw logs with one command.
- Claims never exceed what the analysis supports.
- Novelty section survives a search done in the final week.

Main risks for this Part

- Someone publishes first. Mitigation: re-search every 2 weeks and target a workshop first for speed.
- Result is null. Mitigation: negative results are reported; the harness and catalogue still stand.
- Tuning on evaluation tasks. Mitigation: pilot-only tuning, hashes, held-out run once.

4. Joint milestones, go/no-go gate, backup study

When	Milestone	Who	Exit condition
End of Week 1	Environment and contracts ready	A, B, C	Lean and VERINA build; models serve deterministically; interfaces signed; novelty matrix v1
End of Week 2	Pilot and go/no-go	All	Section 4 criteria met, or backup chosen; prereg frozen
End of Week 3	Full study ready	All	60 to 100 tasks, driver v1, dry run clean
End of Week 4	Main runs complete	C leads	Raw results for primary models
End of Week 5	Analysis complete	C leads, A supplies RQ3	Tables and figures drafted
End of Week 6	Robustness and half the paper	All	Held-out run done; novelty re-checked
End of Week 7	Submission-ready	C leads	Paper v1 and replication package

Go/no-go criteria (end of Week 2)

Continue with the main study only if all four hold on the 20 pilot tasks:

1. The tactic list proves at least about 40 percent of correct reference solutions (automation is viable). If lower, widen the task filter or tactic list on the pilot only.
2. C repairs at least one task that A and B both fail on, with a hand-verified proof.
3. Integrity checks run cleanly with no false successes.
4. Estimated cost for the full study fits the available compute.

Backup studies (same infrastructure, nothing wasted)

Option	Study	Output
Backup 1	How far can fixed automation go? Characterise which VERINA solutions Lean tactics can prove and which code features block them, plus a small-model rewrite experiment.	Empirical study and dataset
Backup 2	Proof-friendliness benchmark: reference versus mutated versus LLM-rewritten code, scored by automation success.	Benchmark and analysis
Backup 3	B-style proof repair across model sizes 1.5B to 7B with a hint ablation.	Scaling study

5. Risks, open decisions, publication strategy

Risk	Mitigation	Owner
Tactics prove too few programs	Week 2 pilot; backup study; widen list on pilot only	A
Mutants unrealistic	Add real small-model errors; report both	B
C wins through degenerate code or spec-gaming	Locked spec and tests, region diff, 10 percent hand audit	A and C
Unfair budgets	Equal N model calls; also report tokens and wall clock	C
Tuning on evaluation tasks	Pilot-only tuning, frozen hashes, held-out split run once	C
Quantization or version drift	Pin model hashes and Lean toolchain; log everything	A and B
Compute limits	Start at 60 tasks; GPU server only for 7B; never sudo reboot the server	B
Someone publishes first	Re-search every 2 weeks; workshop-first strategy	C
Small models fail at code repair itself	Report compile rate; T upper bound attributes failures	B and C
One member blocked or absent	Interfaces are stubbed in Week 1; any member can run the other's call from the README	All

Open decisions

1. Hard paper deadline (this plan assumes Week 7 ends on 2026-12-18).
2. Compute: laptop only, or the GPU server as well.
3. Bug source: mutants only, or mutants plus real model errors.
4. Target venue, which sets the page limit.
5. Whether the professor expects a pre-registration document submitted anywhere.
6. Verify the VERINA headline figures. Search snippets reported 61.4 percent correct code and 3.6 percent valid proofs for o4-mini; an earlier note said 72.6 percent and 4.9 percent. Confirm against the paper before quoting.
7. Who is Member A, B, and C (assign by skills in section 3).

Publication strategy

- First target: a workshop paper (LLM4Code or FormaliSE).
- If C clearly beats B on the main and held-out sets, aim for ASE NIER or the main track.
- A clean negative result is still publishable and tells the community that small models need proof skills.